GUÍA OFICIAL DE IMPLEMENTACIÓN MLOPS

# Despliegue Continuo de Modelos de Machine Learning con Docker, Azure y GitHub Actions

Paso a paso para clonar un proyecto, contenerizar con Docker, desplegar en Azure Container Registry & Web App, y automatizar cambios con CI/CD.


| | |
|---|---|
| **Estudiante:** | Rafael Chamorro |
| **Institución:** | Universidad Icesi |
| **Suscripción Cloud:** | Azure for Students |
| **Repositorio GitHub:** | https://github.com/rafaelchamorro/sample-pycaret |
| **Stack Tecnológico:** | PyCaret, Flask, Docker, Azure ACR, Web App for Containers, GitHub Actions |

## 📑 Contenido de la Guía


- **Módulo 1:** Duplicación y Preparación del Repositorio en GitHub
- **Módulo 2:** Contenerización con Docker y Validación de Calidad (Quality Gate)
- **Módulo 3:** Aprovisionamiento y Configuración de Recursos en Azure for Students
- **Módulo 4:** Construcción del Pipeline CI/CD en GitHub Actions
- **Módulo 5:** El Reto del Taller: Actualización Automática del Título a Español
- **Módulo 6:** Verificación en Producción y Control de Créditos en la Nube

# Módulo 1: Duplicación y Preparación del Repositorio

El proyecto base proviene del repositorio del curso ( `avila196/sample-pycaret` ). Para trabajar sobre tu propia cuenta sin requerir permisos en el repositorio original, se realiza una migración limpia de origen remoto.

### Paso 1.1: Clonar el proyecto localmente

```
# Clonar el proyecto base en tu máquina
git clone https://github.com/avila196/sample-pycaret.git
cd sample-pycaret
```

### Paso 1.2: Crear el repositorio vacío en tu cuenta de GitHub

1. Ingresa a tu cuenta en `https://github.com/new` .
2. Define el **Repository name**: `sample-pycaret` .
3. Selecciona la visibilidad (Público o Privado).
4. **Crítico:** Deja **desmarcadas** las casillas de *Add a README file*, *.gitignore* y *License* para que el repositorio se cree completamente vacío.
5. Haz clic en **Create repository**.

### Paso 1.3: Redirigir el origen remoto a tu cuenta personal

```
# Cambiar el origen remoto al de tu cuenta personal
git remote set-url origin https://github.com/rafaelchamorro/sample-pycaret.git

# Verificar que los enlaces apunten a tu usuario
git remote -v
# origin  https://github.com/rafaelchamorro/sample-pycaret.git (fetch)
# origin  https://github.com/rafaelchamorro/sample-pycaret.git (push)
```

# Módulo 2: Contenerización con Docker y Quality Gate

En MLOps, empaquetar el código junto con el archivo de modelo serializado ( `deployment_20260516.pkl` ) garantiza que el servicio responda de forma idéntica en local y en la nube.

### Paso 2.1: El Dockerfile Optimizado para Producción

Se modificó el archivo `Dockerfile` para asegurar compatibilidad con Azure Web App:

```
FROM python:3.11

# Configuración del entorno virtual aislado
RUN pip install virtualenv
ENV VIRTUAL_ENV=/venv
RUN virtualenv /venv -p python3
ENV PATH="$VIRTUAL_ENV/bin:$PATH"

# Puerto dinámico de producción
ENV PORT=5000

WORKDIR /app
ADD . /app

# Instalación de librerías del modelo (PyCaret, Flask, pandas)
RUN pip install -r requirements.txt

EXPOSE 5000

# Comando de arranque del servidor web
CMD ["python", "app.py"]
```

### Paso 2.2: Soporte para Puerto Dinámico en app.py

Azure inyecta dinámicamente la variable de entorno `PORT` . En `app.py` se garantiza la lectura de esta variable:

```
if __name__ == '__main__':
    import os
    port = int(os.environ.get("PORT", 5000))
    debug_mode = os.environ.get("FLASK_DEBUG", "false").lower() == "true"
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

### Paso 2.3: Validación Automatizada del Modelo (Pytest Quality Gate)

Antes de gastar tiempo y dinero en construir imágenes en la nube, se ejecutan pruebas automáticas en `tests/test_model.py` :

```
from pycaret.regression import load_model, predict_model
import pandas as pd

def test_model_loading_and_inference():
    # 1. Carga el artefacto serializado
    model = load_model('deployment_20260516')
    assert model is not None

    # 2. Inferencia con datos de prueba
    sample_data = pd.DataFrame([{
        'age': 35, 'sex': 'male', 'bmi': 26.5,
        'children': 2, 'smoker': 'no', 'region': 'northeast'
    }])
    pred = predict_model(model, data=sample_data)
    assert 'prediction_label' in pred.columns
    assert float(pred.loc[0, 'prediction_label']) > 0
```

# Módulo 3: Infraestructura en Azure for Students

Con tu suscripción de **Azure for Students**, se crearon los siguientes componentes sin costo real:

| Recurso de Azure | Nombre | Región | Plan / Nivel | Propósito |
|---|---|---|---|---|
| **Resource Group** | `modeldeploy` | Brazil South | - | Agrupa el registro de imágenes. |
| **Container Registry (ACR)** | `pycaret1` | Brazil South | Standard (o Basic) | Almacena de forma privada las imágenes Docker ( `pycaret1.azurecr.io` ). |
| **Web App (App Service)** | `pycaret1` | West US 3 | B1 Linux Container | Ejecuta el contenedor en producción en la nube. |




Figura 1: Vista general de los recursos de Azure en Brazil South y West US 3.




Figura 2: Panel principal de Azure Container Registry con el servidor 'pycaret1.azurecr.io'.

## Paso 3.1: Habilitar el Usuario Administrador en ACR

Para que GitHub Actions pueda hacer `docker login` en tu registro, debes habilitar el usuario admin:

```
az acr update -n pycaret1 --admin-enabled true
```

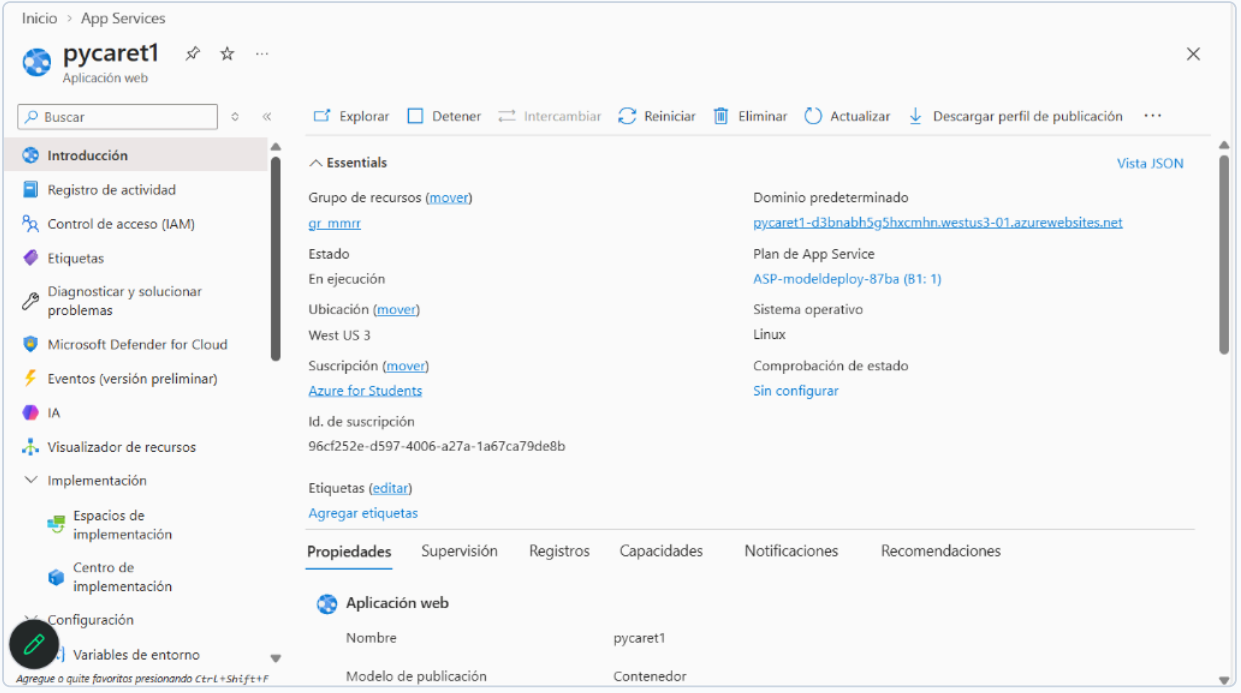



Figura 3: Panel de control de la Azure Web App for Containers 'pycaret1'.

## Paso 3.2: Las 2 Configuraciones Críticas en Azure Web App

> ⚠️ **Imprescindible para evitar errores de despliegue:**

1. **Configurar el Puerto del Contenedor ( `WEBSITES_PORT = 5000` ):**
   En tu Web App en Azure Portal, entra a **Configuración > Variables de entorno** y agrega: `WEBSITES_PORT = 5000` . Esto indica a Azure que envíe el tráfico al puerto donde escucha Flask.
2. **Permitir Credenciales Básicas SCM (Publish Profile):**
   Azure bloquea la publicación con Publish Profile por defecto en cuentas nuevas. Se activa con este comando de Azure CLI:

```
az resource update --resource-group gr_mmrr --namespace Microsoft.Web --resource-typ
```

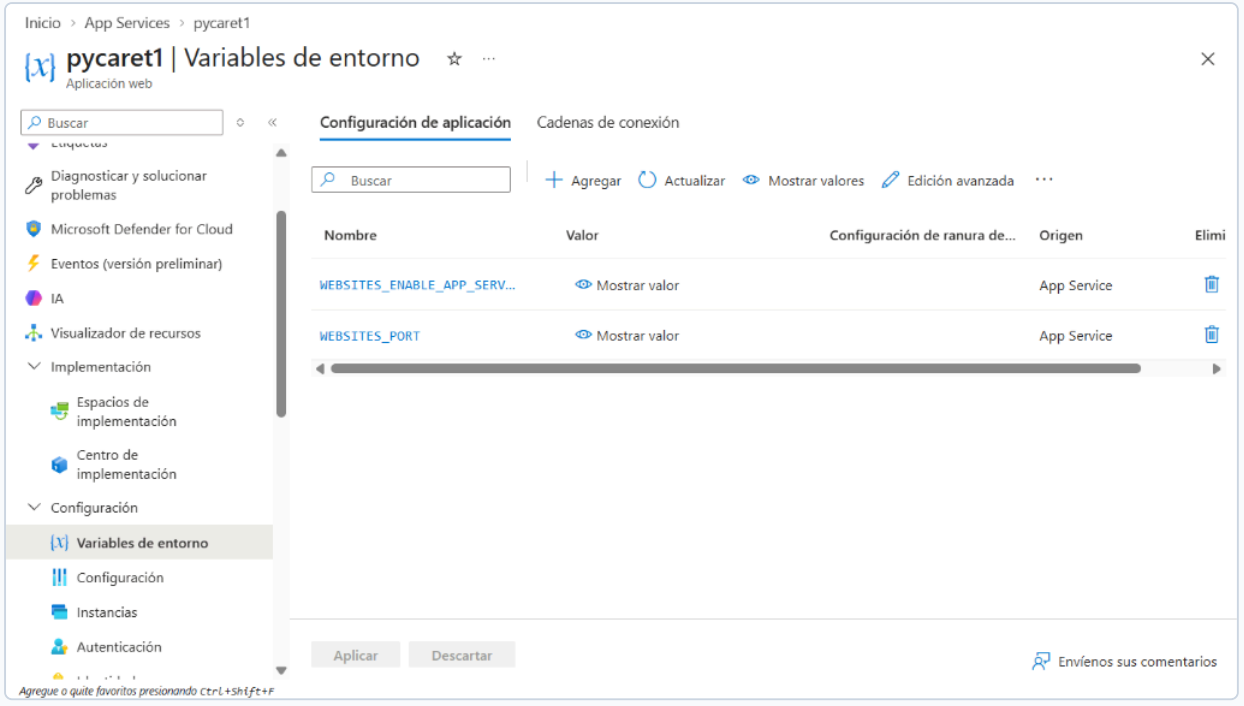



Figura 4: Variable de entorno WEBSITES_PORT = 5000 guardada y aplicada en Azure.

# Módulo 4: Pipeline CI/CD en GitHub Actions

En tu repositorio en GitHub (**Settings > Secrets and variables > Actions**), se registran los 5 secretos:

| Secreto de GitHub | Valor Configurado | Origen |
|---|---|---|
| `ACR_LOGIN_SERVER` | `pycaret1.azurecr.io` | ACR > Información General |
| `ACR_USERNAME` | `pycaret1` | ACR > Claves de acceso |
| `ACR_PASSWORD` | *[Contraseña de Administrador]* | ACR > Claves de acceso |
| `AZURE_WEBAPP_NAME` | `pycaret1` | Nombre de la Web App |
| `AZURE_WEBAPP_PUBLISH_PROFILE` | *[Contenido XML completo]* | Web App > Botón "Descargar perfil de publicación" |

### El Workflow Completo (.github/workflows/deploy.yml)

```
name: MLOps CI/CD - PyCaret Azure Pipeline

on:
  push:
    branches: [ main ]
  workflow_dispatch:

env:
  PYTHON_VERSION: "3.11"
  IMAGE_NAME: "insurance-app"

jobs:
  # 1. QUALITY GATE
  test-and-validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}
          cache: "pip"
      - run: |
          pip install -r requirements.txt
          pip install -r requirements-test.txt
      - name: Validar Modelo y Endpoints
        env:
          PYTHONPATH: .
        run: python -m pytest tests/ -v

  # 2. DOCKER BUILD & PUSH A ACR
  build-and-push:
    needs: test-and-validate
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # ═══════════════════════════════════════════════════════════
      # RETO: CAMBIO AUTOMÁTICO DEL TÍTULO A ESPAÑOL POR WORKFLOW
      # ═══════════════════════════════════════════════════════════
      - name: Asegurar título de la página en Español
        run: |
          sed -i 's/Predict Health Insurance Bill - V2/Predicción del Costo del Seguro

      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ${{ secrets.ACR_LOGIN_SERVER }}
          username: ${{ secrets.ACR_USERNAME }}
          password: ${{ secrets.ACR_PASSWORD }}

      - name: Construir y Publicar Imagen en Azure ACR
        uses: docker/build-push-action@v5
        with:
```

```
          context: .
          push: true
          tags: |
            ${{ secrets.ACR_LOGIN_SERVER }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
            ${{ secrets.ACR_LOGIN_SERVER }}/${{ env.IMAGE_NAME }}:latest
          cache-from: type=gha
          cache-to: type=gha,mode=max

  # 3. DESPLIEGUE CONTINUO EN AZURE WEB APP
  deploy-to-azure:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - uses: azure/webapps-deploy@v3
        with:
          app-name: ${{ secrets.AZURE_WEBAPP_NAME }}
          publish-profile: ${{ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }}
          images: "${{ secrets.ACR_LOGIN_SERVER }}/${{ env.IMAGE_NAME }}:${{ github.sha

      - name: Smoke Test Post-Despliegue
        run: |
          sleep 25
          curl -f -s https://${{ secrets.AZURE_WEBAPP_NAME }}.azurewebsites.net/health
```

# Módulo 5: Verificación en Producción

Una vez ejecutado el workflow tras un `git push origin main`:

> ✅ **Resultado en Producción:**
> Al ingresar a tu URL pública:
> **https://pycaret1-d3bnabh5g5hxcmhn.westus3-01.azurewebsites.net**
> La página muestra el título actualizado: **"Predicción del Costo del Seguro Médico"**, demostrando que el pipeline construyó la imagen, la subió al ACR y actualizó la Web App sin requerir ninguna acción manual adicional.

### Consumo de la API REST de Producción (cURL):

```
curl -X POST https://pycaret1-d3bnabh5g5hxcmhn.westus3-01.azurewebsites.net/predict_api
  -H "Content-Type: application/json" \
  -d '{"age": 35, "sex": "male", "bmi": 26.5, "children": 2, "smoker": "no", "region":

# Respuesta: {"output": 4350}
```

# Módulo 6: Control de Créditos en Azure for Students

- **GitHub Actions:** Es 100% gratuito e ilimitado para repositorios públicos.
- **Azure for Students:** Te otorga $100 USD de saldo que no toca cuentas bancarias ni tarjetas.
- **Optimización de Costos:**
  - Puedes cambiar tu ACR a SKU básico cuando quieras: `az acr update -n pycaret1 --sku Basic`.
  - Cuando termines tu entrega, puedes entrar a Azure Portal y hacer clic en **Detener (Stop)** en tu Web App para que no consuma saldo de cómputo en horas ociosas.